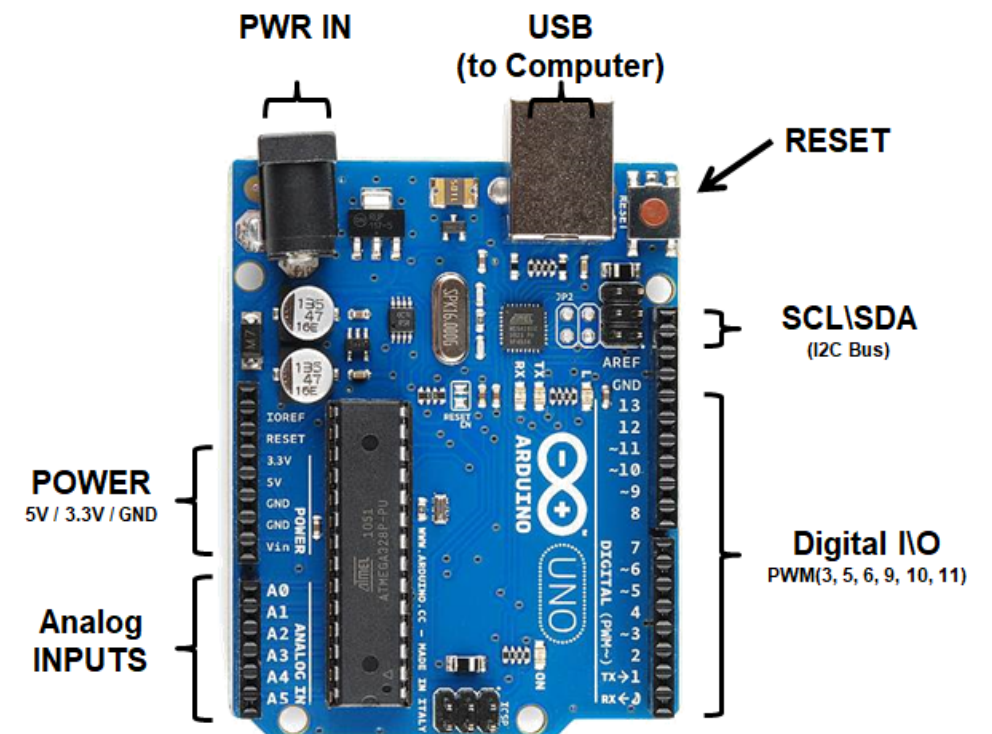


EG1311: Design & Make

Digital Computers

Dr. Jason S. Ku

Digital Computer: What is it?

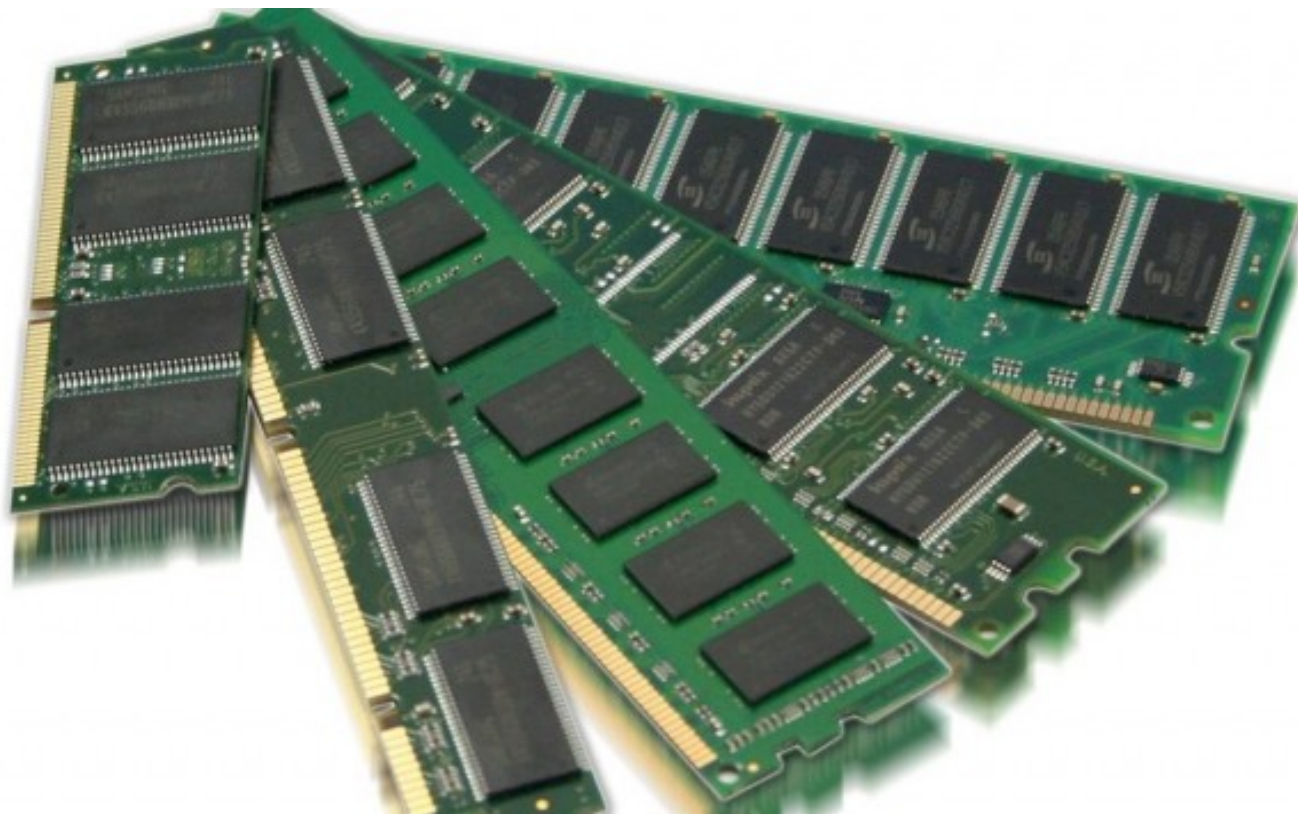
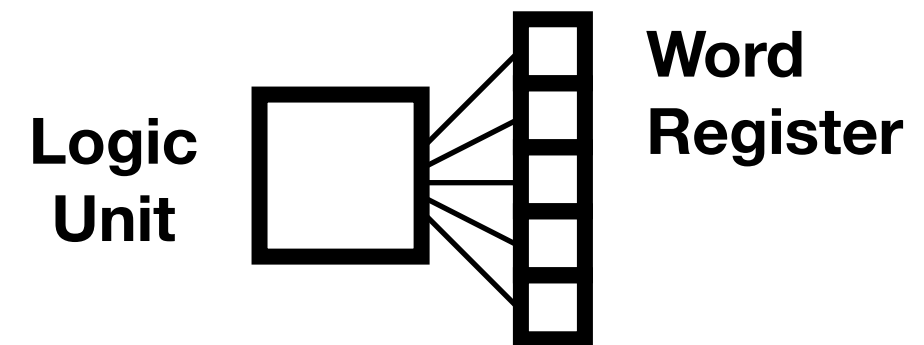
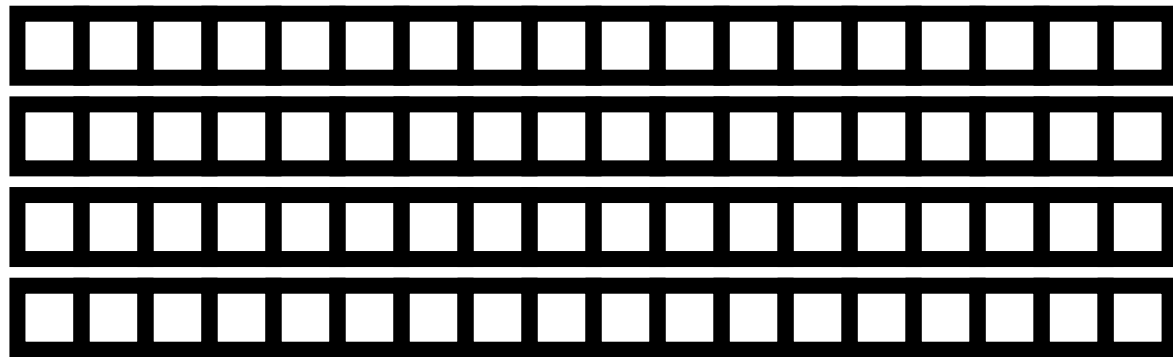


Digital Computer: Components

Memory (Storage)

CPU (Brain)

Byte Addressable



Digital Computer: Binary

- Like base-10, binary most significant on left
- 1 byte = 8 bits **[0, 255]**
- Machine code (assembly) instruction set
 - Data Transfer (read and write memory)
 - Data Manipulation (change memory)
 - Program Flow (change next execution)

Processor	On-Board Memory	Word Size	Registers	Instructions	Processor Speed
ATmega328P	32 KBs	8 bits	32	131	~ 1 MIPS
Intel i7	16,000 KBs	64 bits	16	1503	~ 100,000 MIPS

Programming: C

- Higher level language than assembly
- One line can be many assembly instructions
- Written in text rather than bytes (**ASCII**)
- Compiler converts to assembly

Binary	Dec	ASCII	Binary	Dec	ASCII	Binary	Dec	ASCII	Binary	Dec	ASCII	Binary	Dec	ASCII	Binary	Dec	ASCII	Binary	Dec	ASCII	Binary	Dec	ASCII
00000000	0	NUL	00010000	16	DLE	00100000	32	space	00110000	48	0	01000000	64	@	01010000	80	P	01100000	96	`	01110000	112	p
00000001	1	SOH	00010001	17	DC1	00100001	33	!	00110001	49	1	01000001	65	A	01010001	81	Q	01100001	97	a	01110001	113	q
00000010	2	STX	00010010	18	DC2	00100010	34	"	00110010	50	2	01000010	66	B	01010010	82	R	01100010	98	b	01110010	114	r
00000011	3	ETX	00010011	19	DC3	00100011	35	#	00110011	51	3	01000011	67	C	01010011	83	S	01100011	99	c	01110011	115	s
00000100	4	EOT	00010100	20	DC4	00100100	36	\$	00110100	52	4	01000100	68	D	01010100	84	T	01100100	100	d	01110100	116	t
00000101	5	ENQ	00010101	21	NAK	00100101	37	%	00110101	53	5	01000101	69	E	01010101	85	U	01100101	101	e	01110101	117	u
00000110	6	ACK	00010110	22	SYN	00100110	38	&	00110110	54	6	01000110	70	F	01010110	86	V	01100110	102	f	01110110	118	v
00000111	7	BEL	00010111	23	ETB	00100111	39	'	00110111	55	7	01000111	71	G	01010111	87	W	01100111	103	g	01110111	119	w
00001000	8	BS	00011000	24	CAN	00101000	40	(	00111000	56	8	01001000	72	H	01011000	88	X	01101000	104	h	01111000	120	x
00001001	9	HT	00011001	25	EM	00101001	41	)	00111001	57	9	01001001	73	I	01011001	89	Y	01101001	105	i	01111001	121	y
00001010	10	LF	00011010	26	SUB	00101010	42	*	00111010	58	:	01001010	74	J	01011010	90	Z	01101010	106	j	01111010	122	z
00001011	11	VT	00011011	27	ESC	00101011	43	+	00111011	59	;	01001011	75	K	01011011	91	[	01101011	107	k	01111011	123	{
00001100	12	FF	00011100	28	FS	00101100	44	,	00111100	60	<	01001100	76	L	01011100	92	\	01101100	108	l	01111100	124	
00001101	13	CR	00011101	29	GS	00101101	45	-	00111101	61	=	01001101	77	M	01011101	93	]	01101101	109	m	01111101	125	}
00001110	14	SO	00011110	30	RS	00101110	46	.	00111110	62	>	01001110	78	N	01011110	94	^	01101110	110	n	01111110	126	~
00001111	15	SI	00011111	31	US	00101111	47	/	00111111	63	?	01001111	79	O	01011111	95	_	01101111	111	o	01111111	127	DEL

Programming: Variables & Types

- **Variable:** text name for address in memory
- **Type:** organization and size of memory
- Variables must be declared before use

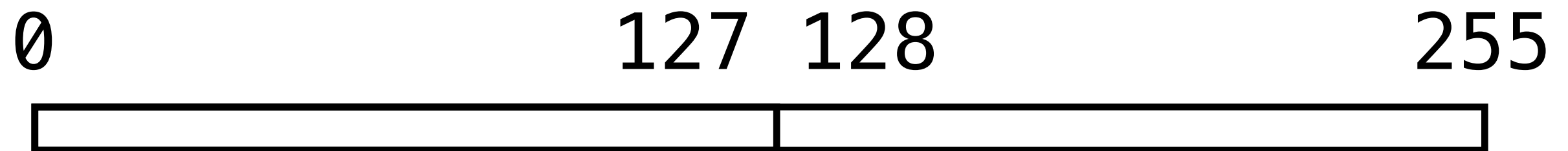
Comments

```
char a;           // 8 bits
int b;            // 16 bits
unsigned long u;  // 32 bits
float c;          // 32 bits
```

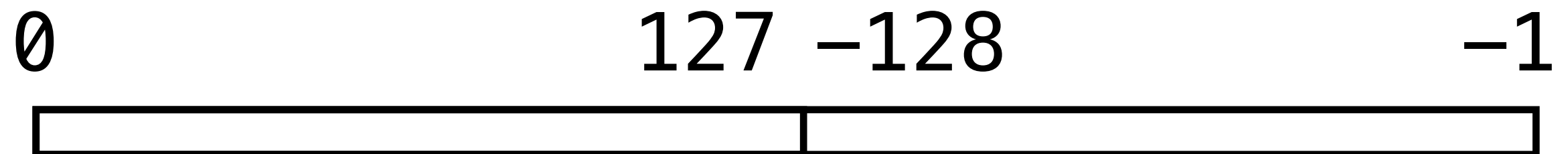
Programming: Variables & Types

8 bits

unsigned char



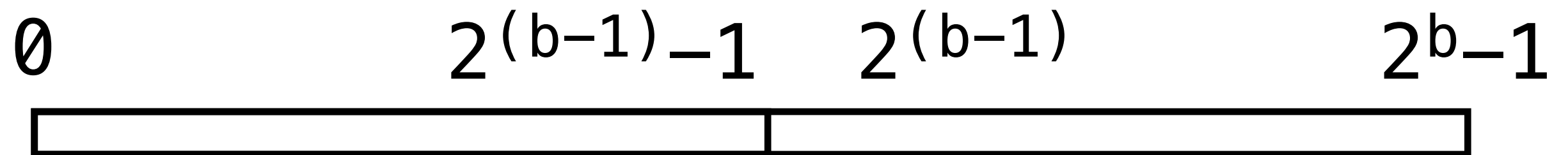
char



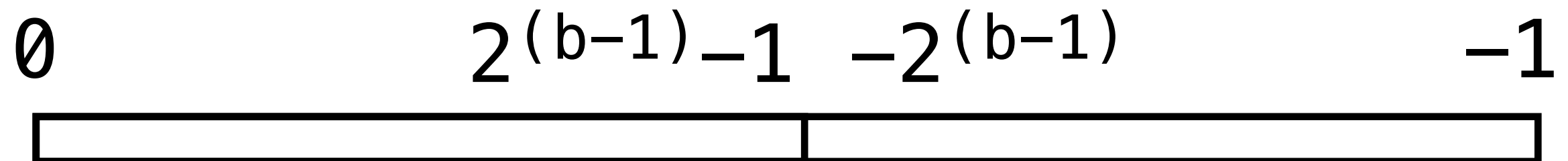
Programming: Variables & Types

$b = \# \text{ bits in type}$

unsigned (type)



(type)



Programming: Assignment

- Assignment denoted by equal sign (=)
- Can declare and assign on same line
- All statements end with semicolon (;)

```
a = 'a';           // assign    97
```

```
int d = a;         // declare & assign
```

```
int e = 128;       // 128
```

```
char f = e;        // -128
```

```
128 = a;           // ERROR
```

Programming: Own Types

- **Struct** and **Class** Objects

```
struct Student {    // define
    int age;
    int grade;
};
```

```
Student jason;      // declare
```

```
jason.age = 37;     // assign
```

```
jason.grade = 83;  // assign
```

Programming: Operators

- Manipulate one or two numbers

- Arithmetic $+$ $-$ $*$ $/$ $\%$

- Comparison $==$ $!=$ $<$ $<=$ $>$ $>=$

- Logic $!$ $\&\&$ $||$ 1010 10

- Bitwise $\&$ $|$ $^$ $\sim$ $<<$ $>>$ 1001 9

$\&\&$	T	F
T	T	F
F	F	F

$ $	T	F
T	T	T
F	T	F

9 $\&$ 10 = 1000 8

9 $|$ 10 = 1011 11

9 $^$ 10 = 0011 3

6 $<<$ 1 = 0110b $<<$ 1 = 1100b = 12

Programming: Operators

- Manipulate one or two numbers
 - Arithmetic `+` `-` `*` `/` `%`
 - Comparison `==` `!=` `<` `<=` `>` `>=`
 - Logic `!` `&&` `||`
 - Bitwise `&` `|` `^` `~` `<<` `>>`

```
int g = 6 + 12; // 18
```

```
int h = (g == 5) || (g < 5); // 0
```

Practice

```
int a = 14;
```

```
int b = 'a';
```

```
b = a - b; // ?
```

What number is stored
in variable b?

A

B

C

D

A) -111

B) -83

C) 83

D) 111

Practice

```
int c = (1 << 15) - 1; // ?
```

What number is stored in variable c?

A

B

C

D

A) 32766

B) 32767

C) 32768

D) 32769

Practice

```
int c = (1 << 15) - 1; // 32767  
c = c + 1;           // ?
```

What number is stored in variable c?

A

B

C

D

A) 32768

B) 0

C) -32767

D) -32768

Practice

```
int d = (! (10 & 6)) || ('a' % 3);  
// ?
```

What number is stored in variable d?

A

B

C

D

A) 0

B) 1

C) 2

D) 3

EG1311: Design & Make

Digital Computers

Dr. Jason S. Ku